

Система автоматизированного обогащения товарных данных для интернет-магазина

Фролов М. А.

Московский авиационный институт
Институт № 8, кафедра 806 «Вычислительная математика и программирование»
Научный руководитель: Судаков В. А., д. т. н., профессор кафедры 806

Москва, 2026

Две стороны проблемы обогащения

Человек не масштабируется

- ▶ Каталоги — десятки миллионов SKU
- ▶ Партнёры присылают только базовые поля
- ▶ Заполнение 50–150 атрибутов вручную = узкое место
- ▶ Каталог хранит, но не извлекает атрибуты

Машина-в-лоб слишком дорого

- ▶ Прямой вызов БЯМ на каждый SKU
- ▶ На 10^7 SKU — неприемлемая стоимость
- ▶ Точность vs цена — компромисс
- ▶ Нет контроля качества «на крае»

⇒ Нужно дешёвое решение, не теряющее качество — **поатрибутная маршрутизация** между слоями разной стоимости.

Аналоги: пробел в существующих подходах

Критерий	TXtract	MAVE	FrugalGPT	AutoMix	Наша система
Поатрибутная политика	✓	✓	—	—	✓
Калиброванный отказ от ответа	—	—	—	✓	✓
Маршрутизация по стоимости	—	—	✓	✓	✓
Контроль на непересекающихся товарах	—	—	—	—	✓
Гибридная архитектура слоёв	—	—	—	—	✓
Локальное развёртывание	частично	частично	—	—	✓

Ни одна из существующих систем не объединяет **поатрибутную политику**, **контроль на новых товарах** и **гибридную архитектуру** одновременно.

TXtract / MAVE (Amazon / Google): требуют GPU и трансформерной инфраструктуры.

Цель, задачи и рабочая гипотеза

Цель. Программное средство обогащения товарных данных на основе гибридной каскадной архитектуры.

Задачи

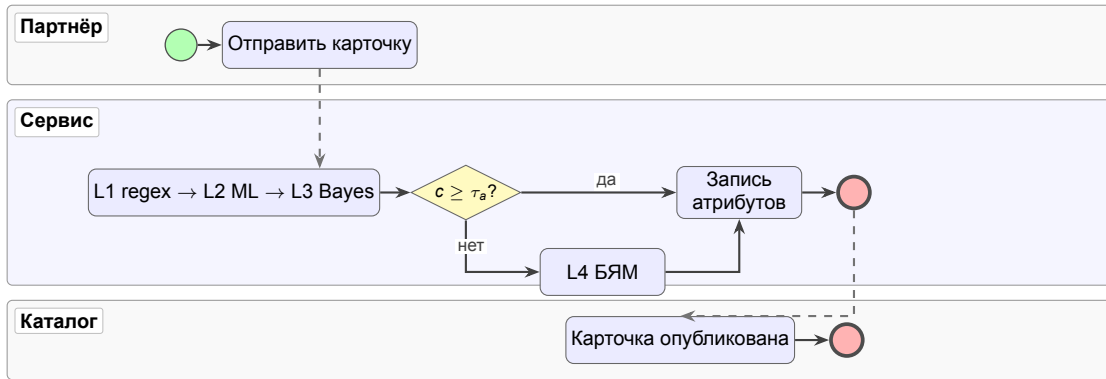
1. Спроектировать гибридный каскад из 5 слоёв с поатрибутной политикой.
2. Реализовать слои как независимо обновляемые компоненты.
3. Подготовить эталонную разметку и контрольную выборку без пересечения кодов.
4. Оценить точность, покрытие и стоимость; сравнить с прямой БЯМ.
5. Реализовать программный комплекс с REST API и веб-интерфейсом.

Гипотеза. Каскад $\text{regex} \rightarrow \text{ML} \rightarrow \text{Bayes} \rightarrow \text{LLM}$ с поатрибутной калиброванной эскалацией обеспечивает заданную точность при кратном сокращении стоимости.

Объект: процессы обогащения карточек товаров.

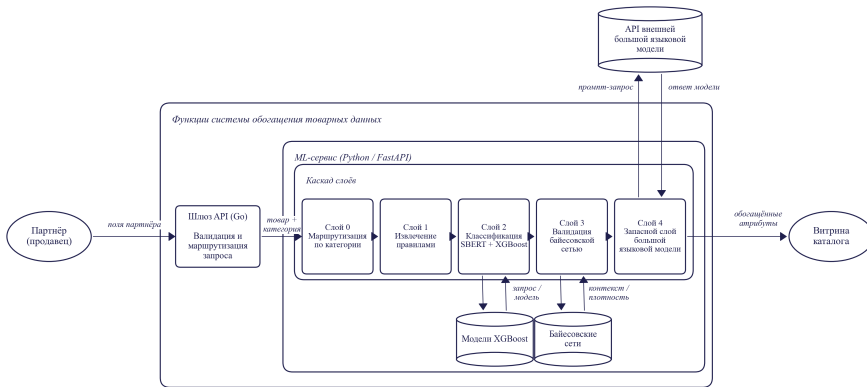
Предмет: методы извлечения атрибутов и политики маршрутизации между слоями.

Процесс обогащения карточки: BPMN



○ старт, ○ финал, задача, ◇ XOR-шлюз, — управление, - - сообщения

Архитектура системы (ГОСТ Р 57100-2016)



Партнёр → шлюз API → ML-сервис (каскад L0–L4) → витрина каталога.

Принципы: дешёвое до дорогого; калибровка τ_a ; явный отказ при $c < \tau_a$.

Что делает каждый слой

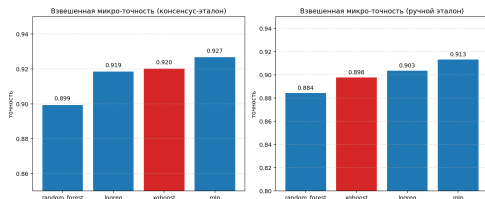
21 классификатор по парам «категория-атрибут» (8/6/7); бюджет LLM $\beta = 10\%$, фактически **4,1 %**.

Слой	Что делает	Когда не справляется	Доля
L0 router	XGBoost на эмбединге $\rightarrow$ категория + OOD-флаг	OOD $\Rightarrow$ слой 4 «как есть»	—
L1 regex	Прецизионные поатрибутные шаблоны	Нет матча $\Rightarrow$ слой 2	21,9 %
L2 ML	XGBoost на 768-мерном эмбединге; Platt + isotonic	$p_{\max} < \tau_a \Rightarrow$ слой 3	73,2 %
L3 Bayes	Валидатор плотностей $P_B(\hat{y} e)$	$P_B < \theta_a \Rightarrow$ слой 4	0,8 %
L4 LLM	gpt-oss-120b / Sonnet 4.5 / Gemini Flash	— (последний слой)	4,1 %

$\Rightarrow$ 95 % ячеек закрываются дешёвыми слоями; система гибкая — слой 2 заменяется без правки ядра.

Слой 2: сравнение алгоритмов классификации

Четыре алгоритма на общем эмбединге; выбор XGBoost обоснован **калибровкой**, а не максимумом точности — система гибкая, классификатор сменяется конфигом.



Cells-weighted micro-асс: четыре метода в пределах ± 1 пп

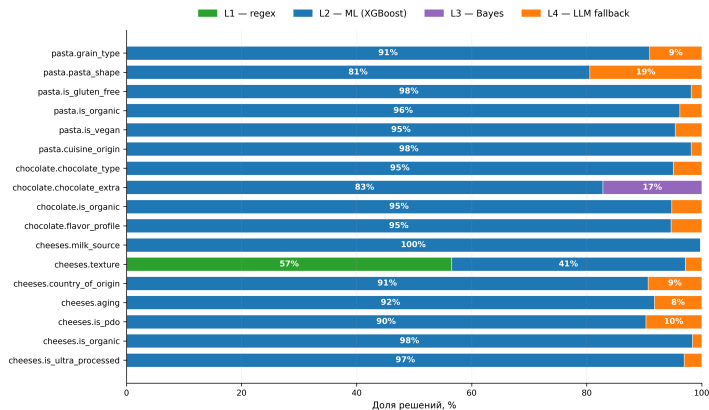
Метод	Асс	F1	ECE	t , с
XGBoost	92,02 %	0,849	0,026	1062
MLP	92,67 %	0,857	0,031	54
LogPer	91,85 %	0,836	0,035	8
Random Forest	89,94 %	0,809	0,027	523

Почему XGBoost (а не MLP/RF/нейросеть):

- ▶ MLP: +0,65 пп асс, но ECE **0,031** $\Rightarrow$ сдвинутые $p_{\max}$,
- ▶ ЛогПер / RF: ECE 0,035 / точность — 2 пп,
- ▶ **XGBoost**: ECE **0,026** $\Rightarrow \tau_a$ корректно калиброваны.

§3.3.7.5; multi-output (одна модель на 2 атрибута) — B19.

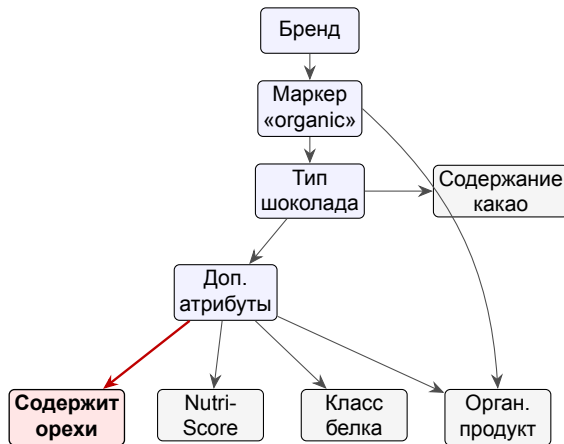
Какой слой заполняет какие атрибуты



Исключены 3 тривиальных атрибута (100 % L1 regex): `pasta.is_filled`, `chocolate.is_filled`, `chocolate.contains_nuts`

3 атрибута исключены ($\geq 99\%$ L1 regex): `pasta.is_filled`, `chocolate.is_filled`, `chocolate.contains_nuts`.

Bayes — валидатор, не классификатор



1. Что построено.

pgmpy 1.1.2: структура — Hill Climb + BIC; CPT — `BayesianEstimator` с `BDeu`.

2. Роль — валидатор.

Самостоятельная точность $\approx 52\%$ (ниже базовой), но плотность $P_B(\hat{y} | \epsilon)$ на нижнем 5-перцентиле даёт сигнал отбраковки.

3. Где включён.

Точноно: **1 пара из 21** — `chocolate / contains_nuts`:
+6,6 пп точности при **+5,3 пп** расходов запасного слоя.

DAG, chocolate (Hill Climb + BIC, BDeu CPT) — активный валидатор

Данные и протокол оценки

Источник. Open Food Facts, 4,5 млн SKU (~40 % FR / 10 % EN / 7 % DE).

3 категории. Паста (8 атр.), шоколад (6), сыры (7) — **21 пара** «категория-атрибут».

Эталон. Два уровня: основной консенсус БЯМ ($n = 16\,360$) + проверочный ручной ($n = 566$, нижняя оценка).

Протокол. Разбивка без пересечения товарных кодов; предрегистрация `cd9ac7a` (2026-05-13). Порог уверенности τ_a подобран на валидационной, не на тестовой выборке.

Без отдельных моделей на язык: разброс точности по основным языкам (~40 % FR / 10 % EN / 7 % DE)

Категория	Атрибутов
Паста	8
Шоколад	6
Сыры	7
<i>Холодный старт:</i>	
Электроника	8

Главный результат

95,5 %

точность каскада, ДИ Уилсона [95,2; 95,8]

93,0 %

сквозная (производственная) точность

Метрика	Значение	<i>n</i>
Макро-F1 каскада (взвеш. по ячейкам)	0,890	16 360
Макро-precision / макро-recall	0,918 / 0,881	16 360
Точность маршрутизатора категории	97,2 %	570
Покрытие сквозной цепочки	97,3 %	16 360
Нижняя оценка на ручной разметке	86,7 %	566

Выборка без пересечения товарных кодов. **Достоверность:** МакНемара $p < 0,001$ (Бонферрони, $k=21$); ДИ Уилсона 95 % на всех headline-числах. Подробнее — В17.

Разбивка по категориям и атрибутам

Эталон — LLM-consensus, $n = 16\,360$ ячеек; agg-метрики взвешены по числу ячеек.

Категория	Ячеек	Точность	Макро-F1	Макро-prec	Макро-recall
Паста (8 атр.)	7 593	95,8 %	0,883	0,930	0,869
Шоколад (6 атр.)	6 475	94,9 %	0,894	0,902	0,893
Сыры (7 атр.)	2 292	95,8 %	0,897	0,920	0,883
Всего	16 360	95,5 %	0,890	0,918	0,881

Диапазон по 21 атрибуту: $\uparrow$ `cheeses.is_pdo` 99,7 % / F1 0,993 • $\downarrow$ `chocolate.flavor_profile` 87,5 % / F1 0,684 • `pasta.is_gluten_free` 99,2 % / F1 0,978 • `pasta.is_filled` 88,5 % / F1 0,786.

Наблюдение: разброс по категориям ≤ 1 пп точности и $\leq 0,014$ по F1 — каскад одинаково силён на всех трёх доменах. Просадки сосредоточены на 2-3 атрибутах с редкими классами ($n_{class} < 60$); их разбор — B8.

Пример: chocolate / contains_nuts

Атрибут с активным Bayes-валидатором (см. слайд про Bayes); пример работы каскада на конкретной паре «категория-атрибут».

	Предсказание	
	нет	да
Истина	698	51
	6	371

Матрица ошибок, $n = 1126$ ячеек шоколада

Класс	prec	recall	F1
нет (без орехов)	99,1 %	93,2 %	0,961
да (с орехами)	87,9 %	98,4 %	0,929
Итого	micro 94,9 %		0,945

Почему сильный результат:

- ▶ высокая precision на «нет» (99,1 %) — критично для аллергиков,
- ▶ высокий recall на «да» (98,4 %) — все орехи отловлены,
- ▶ Bayes-валидатор перепроверяет 5,3 % подозрительных предсказаний.

Каскад vs прямая БЯМ

«Прямая БЯМ» — одна модель спрашивается на 100 % ячеек без L1/L2/L3. Сравнение на одной выборке ($n = 16\,360$, без пересечения товарных кодов).

Конфигурация	Точность	ДИ Уилсона 95 %	Макро-F1	Стоимость	↓ в
Прямая Sonnet 4.5 (опорная)	83,8 %	[83,2; 84,4]	0,725	1,000	1×
Прямая gpt-oss-120b (открытая)	69,3 %	[68,6; 70,0]	0,561	0,030	33×
Каскад + Gemini 2.5 Flash	93,0 %	[92,6; 93,4]	0,890	0,0017	580×
Каскад + Sonnet 4.5	92,9 %	[92,5; 93,3]	0,888	0,041	24×
Каскад + gpt-oss-120b	90,6 %	[90,1; 91,1]	0,847	0,0012	810×

Главное: F1 каскада **0,890** vs прямая БЯМ **0,725** — каскад выигрывает не только в стоимости, но и в *качестве классификации*.

Вклад слоёв: regex **21,9 %** | ML **73,2 %** | Bayes **0,8 %** | БЯМ **4,1 %**

Средства разработки

Программный комплекс:

- ▶ REST API (FastAPI) + веб-интерфейс
- ▶ Полный каскад L0–L4 за один HTTP-вызов
- ▶ Локальное развёртывание:
`docker-compose up`
- ▶ Возврат по каждому атрибуту: значение, слой-источник, уверенность

Латентность:

~200 мс (каскад) /

< 5 с (эскалация в БЯМ).

Обогащение товарных данных

Каскад `regex` → ML → байес → LLM-fallback. Введите частично заполненный товар — система предскажет недостающие атрибуты.

Ввод товара

Режим определения категории

☒ Авто (классификатор) ☐ Вручную

Категория

Шоколад / десерты

Название товара *

Lindt Excellence Dark 70% Cocoa 100g

Бренд / производитель

Lindt

Состав

cocoa mass, sugar, cocoa butter, soya lecithin, vanilla extract

Количество

100 g

Примеры:

validные: Pasta — Barilla Chocolate — Lindt 70% Cheeses — Roquette

с противоречиями: White vs. 70% cocoa Wheat vs. gluten-free

Обогатить

Результат

Категория:	Авто (router):	OOD:
chocolate	chocolate, p=0.97	нет
Закрито:	LLM-fallback:	Время ответа:
6 / 7 атрибутов	1 атрибут	184 мс

ПРЕДСКАЗАНИЯ ПО АТТРИБУТАМ

АТТРИБУТ	ПРЕДСКАЗАНИЕ	СЛОЙ	УВЕРЕННОСТЬ
cocoa_content_class	70-85 %	<code>regex</code>	1.00
chocolate_type	dark	<code>regex</code>	1.00
contains_nuts	false	<code>ml</code>	0.93
is_organic	false	<code>ml</code>	0.87
contains_milk	false	<code>ml</code>	0.81
flavor_modifier	vanilla	<code>bayes</code>	0.74
filling_type	— будет дозаполнено	<code>llm</code>	—

Слой каскада: `regex` `ml` `bayes` `llm-fallback`

Рисунок 4.1 — Веб-интерфейс демонстрационной системы (схематично): форма ввода и блок результата с отметкой слое-источника и оценкой уверенности

Веб-интерфейс: вход (карточка), результат с уверенностью, trace по

Стек технологий

Сервис обогащения (Python)

- ▶ FastAPI — REST-эндпоинты
- ▶ sentence-transformers (mpnet-base-v2, 50+ языков) — эмбединги L2
- ▶ XGBoost 2.x — классификатор L2
- ▶ pgmpy 1.1.2 — байесовская сеть L3
- ▶ OpenRouter / локальный Ollama — L4

Шлюз и фронтенд

- ▶ Go — шлюз API партнёра
- ▶ React + TypeScript — веб-интерфейс

Данные и оценка

- ▶ PostgreSQL — хранение карточек
- ▶ DuckDB + pyarrow — parquet Open Food Facts
- ▶ pandas, scikit-learn, statsmodels

Развёртывание и воспроизводимость

- ▶ Docker Compose — единая команда запуска
- ▶ Локальный режим (Ollama) — независимость от внешних API
- ▶ Jupyter нотбук — исполняемый §3.3 + ~70 тестов

Влияние на операционную деятельность

580 ×

сокращение стоимости
(комбинированное)

95,5 %

точность каскада
(ДИ Уилсона)

4,1 %

доля языковой модели
(бюджет 10 %)

~200 мс

латентность каскада
(≤5 с с БЯМ)

Оператор каталога — ручная проверка ~3–7 % ячеек вместо 100 %; дашборд качества.

Партнёр-поставщик — API-контракт без изменений; мгновенное обогащение карточки.

Администратор каталога — контроль расходов на БЯМ; единая команда запуска рядом с витриной.

Развитие системы — новые домены через композицию схем; локальный резервный слой.

Разработана в интересах партнёрского интернет-магазина (название по запросу комиссии).

Что нового в работе

Научная значимость.

- ▶ Методика построения гибридного каскада с **поатрибутной калиброванной уверенностью** и **предрегистрируемым** протоколом оценки.
- ▶ Предрегистрированный **отказ** от гипотезы об обучаемом маршрутизаторе в пользу статической политики (простота и предсказуемость).
- ▶ Воспроизводимость методики подтверждена на новой категории (электроника) в режиме холодного старта.

Практическая значимость.

- ▶ Работающее ПО в `docker-compose`, интегрируется рядом с каталогом.
- ▶ Кратное сокращение стоимости и ручной нагрузки контент-менеджеров.
- ▶ Возможность локального развёртывания — независимость от внешних API.

Заключение

Задача	Результат
1. Архитектура каскада	5 слоёв с AND-условием на пороги; явный отказ; идентификация слоя-источника.
2. Реализация слоёв	21 классификатор по 3 категориям (8/6/7); независимое обновление.
3. Эталонная разметка	консенсус БЯМ $n=16\,360$ + ручной $n=566$; разбивка без пересечения кодов.
4. Экспериментальная оценка	каскад 95,5 % / 0,890 макро-F1, сквозная 93,0 % , ручной 86,7 % ; стоимость $\downarrow 24\times$ архит. / $\downarrow 580\times$ комб.
5. Программный комплекс	<code>docker-compose</code> : Go-шлюз + FastAPI + Web-UI; REST; 3 сценария L4.

Итог: точность каскада **95,5 %** / макро-F1 **0,890**, сквозная цепочка **93,0 %** при сокращении стоимости в **24×** раз.

Спасибо за внимание

Точность каскада **95,5 %**,
производственной цепочки **93,0 %**,
сокращение стоимости в **24×** раз.

Исходный код, ноутбук §3.3 в исполняемом
виде, программный комплекс и текст работы
— в репозитории.

Вопросы?



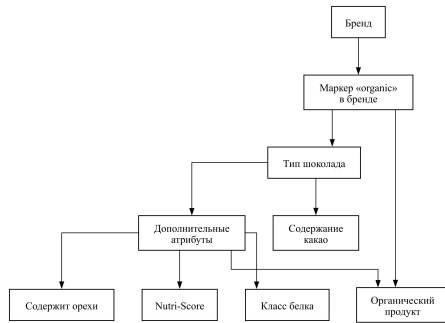
github.com/Qworl/diploma

В2. Байесовский слой как валидатор плотностей

Что построено. Байесовская сеть (Hill Climb + BIC, обучена без пересечения кодов); CPT доучены на объединённой выборке (серебро + золото $\times 10$) с BDeu.

Классификатор отвергнут. Точность на сложном остатке после L2 — $\approx 52\%$, ниже majority. Переориентирован в валидатор: $P(\hat{y} | e)$ ниже 5-го перцентиля $\Rightarrow$ эскалация на L4.

Включение точечное. 1 из 20 пар: chocolate/contains_nuts — **+6,6 пп** точности при **+5,3 пп** расходов. На остальных 19 селективность сигнала $\leq$ базовой ошибки.



В3. Серебряный сигнал vs контрольная разметка

Серебряная разметка (консенсус тегов OFF + 3 языковые модели + слепая разметка Orpus) служит **обучающим** сигналом. Контрольная выборка получена по аудит-циклу.

Цикл «аудит — правки экстрактора — переобучение — измерение» воспроизведён для всех 3 категорий с приростом **+18,2, +16,3, +14,4** пп.

Калибровка уверенности — комбинация Platt + изотонической регрессии поатрибутно; ошибка ECE измеряется на отдельной выборке.

В4. Утечки и борьба с ними

Запрет утечки `categories_tags` в эмбединги.

Эмбединги Слой 2 строятся **только** из партнёрских полей: `product_name + brands + ingredients_text + quantity.categories_tags` — это **выход** обогащения, не вход; использование как признака привело бы к утечке целевой переменной.

Разбиение по непересекающимся товарным кодам (60/20/20; brand overlap train↔eval 82–85%). Бренды в обучающей, валидационной и контрольной выборках не пересекаются — гарантия обобщения на новые бренды.

Анализ по расстоянию k-NN (§3.3.7.3): точность **не падает** на удалённых от обучающей выборки товарах — монотонный рост от Q1 (82,7 %) до Q4 (91,6 %).

В5. Многоязычность

Языковой состав OFF: $\approx 40\%$ FR, 10% EN, 7% DE, остальное — прочие.

Эмбединг: `paraphrase-multilingual-mpnet-base-v2` (50+ языков, архитектура MPNet, общее семантическое пространство).

Поатрибутно-поязыковой срез ($n=3257$ ячеек, предыдущая выборка до extended rerun 2026-05-27 — пересчёт перенесён в перспективу развития темы): взвешенная точность стабильна по основным языкам.

Под балансировку каскад не оптимизировался — это естественный выигрыш многоязычной базовой модели эмбедингов.

В6. Стоимость языковой модели на 1 млн SKU

Каскад делегирует языковой модели лишь 4,1 % ячеек ($n = 17\,062$) — прямая прокси к стоимости. Сокращение затрат **95,9 % (в $24\times$ раз)** относительно «прямой языковой модели на всё».

Распределение по слоям: L1 (regex высокой точности) — 21,9 %, L2 (ML) — **73,2 %**, L3 (regex низкой точности) — 0,8 %, L4 (языковая модель) — 4,1 %.

Проекция на 1 млн SKU × 20 атрибутов ($\sim 2,0 \cdot 10^7$ ячеек):

- ▶ Прямая модель — оплачены все $\sim 2,0 \cdot 10^7$ вызовов.
- ▶ Каскад — оплачены лишь $\sim 8,2 \cdot 10^5$ вызовов (4,1 %).

Топ-3 затратных атрибутов для языковой модели: pasta.grain_type (32,9 %), pasta.pasta_shape (22,8 %), cheeses.is_pdo (13,6 %).

В7. Холодный старт: электроника (§5 ноутбука)

Демонстрация переноса методики на **новую категорию без регулярных правил**.

Электроника (8 атрибутов): смартфоны — RAM, объём памяти, диагональ экрана, поддержка 5G, ...

Перенос: активны только Слой 2 (ML + эмбединги) и Слой 4 (языковая модель); Слой 1 пропускается; Слой 3 — по обучающим парам.

Результат: холодный старт работает на ограниченной обучающей выборке, что подтверждает применимость каскада за пределами продуктового домена.

В8. Где каскад слабее: precision/recall на 3 худших атрибутах

Производственная схема V6, эталон LLM-consensus (extended rerun, n=16 360).

Атрибут	Класс	n	prec	recall	F1
<i>chocolate.flavor_profile</i> — <i>micro</i> 87,5 %, <i>macro-F1</i> 0,684, <i>n</i> =1060					
	intense_bitter	397	91,1 %	98,0 %	0,944
	nutty	281	86,6 %	85,4 %	0,860
	sweet_creamy	190	80,8 %	84,2 %	0,825
	<i>salty_caramel</i> (редкий)	56	90,5 %	67,9 %	0,776
<i>pasta.is_filled</i> — <i>micro</i> 88,5 %, <i>macro-F1</i> 0,786, <i>n</i> =1225					
	false (не фаршированные)	958	87,2 %	100 %	0,931
	<i>true</i> (фаршированные)	267	100 %	47,2 %	0,641
<i>cheeses.texture</i> — <i>micro</i> 89,0 %, <i>n</i> =335					
	hard	109	95,5 %	96,3 %	0,959
	soft	97	91,3 %	97,9 %	0,945
	<i>cream</i> (редкий)	33	73,9 %	51,5 %	0,607

B9. chocolate/contains_nuts — атрибут с активным Bayes-валидатором

Единственная пара из 20, где Bayes-валидатор включён в production (см. B2).
Структура ошибок после фильтрации:

Класс	n	precision	recall	F1
false (без орехов)	749	99,1 %	93,2 %	0,961
true (с орехами)	377	87,9 %	98,4 %	0,929
Сумма	1126	micro 94,9 %		macro-F1 0,945

Почему именно Bayes-валидатор. Атрибут безопасный (аллергия — наличие орехов важно подтверждать жёстко). Распределение асимметричное: false-предсказания *критичны*, true-предсказания допускают перепроверку. Bayes-плотность ловит сомнительные false и эскалирует их в LLM — **+6,6 п. п.** категориальной точности chocolate при **+5,3 п. п.** расходов на LLM.

B10. Macro-F1 на ручной разметке: 0,843 cells-weighted

Консервативная нижняя оценка на независимом эталоне ручной разметки (Claude Opus 4, $n=688$ в-scope, $n=566$ cascade-valid).

Метрика	LLM-consensus ($n=16\,360$)	Ручной gold ($n=566$)
Cascade micro-acc	95,5 %	91,3 %
Cascade macro-F1 (cells-weighted)	0,890	0,843
Macro-precision (cells-weighted)	0,918	0,885
Macro-recall (cells-weighted)	0,881	0,843
Сквозная (производственная) acc	93,0 %	86,7 %

Разрыв 4,2 п. п. (micro-acc) / 0,047 F1 — три причины:
независимый сигнал; меньший $n \rightarrow$ шире ДИ; редкие классы под-представлены (§3.3 ВКР).

В11. На что опираемся (посылки)

1. Унаследованная инфраструктура

Системы управления товарным каталогом хранят атрибуты, но не извлекают их из неструктурированного текста.

2. Теоретическая основа

Многоязычные эмбединги, калиброванная классификация (Platt + isotonic), байесовские сети, prompt-инженерия (few-shot, schema-constrained).

3. Технологический стек

Python / FastAPI / Go-шлюз / React; Docker-compose; XGBoost, sentence-transformers, pgmpy; Ollama для локальной языковой модели.

4. Открытые данные

Open Food Facts (4,5 млн SKU, ~40 % FR / 10 % EN / 7 % DE) + открытые источники по электронике для холодного старта.

В12. Методы исследования

- ▶ **Системный анализ** —
декомпозиция процесса,
выявление противоречий
- ▶ **Управление требованиями** —
ФР / НФР / А, интегральный
критерий Q
- ▶ **Машинное обучение** —
эмбединги + XGBoost,
калибровка Platt + isotonic
- ▶ **Промпт-инженерия** — few-shot
со схемой ответа, селективная
эскалация
- ▶ **Предрегистрируемый
эксперимент** — МакНемара +
Бонферрони, Wilson 95 % CI
- ▶ **Итеративная разработка** —
цикл аудит → правки →
переобучение → измерение

В13. Состав требований

Функциональные (что делает)

- ▶ **F1.** Значение каждого атрибута
- ▶ **F2.** Слой-источник предсказания
- ▶ **F3.** Калиброванная уверенность $c \in [0, 1]$
- ▶ **F4.** Эскалация на запасной слой
- ▶ **F5.** Байес-валидация подозрительных решений
- ▶ **F6.** Поатрибутная настройка через конфиг

Нефункциональные (как работает)

- ▶ **N1.** Точность: $\geq 85\%$ (cascade) / $\geq 90\%$ (+LLM)
- ▶ **N2.** Стоимость: снижение $\geq 2,5\times$ vs прямой LLM
- ▶ **N3.** Производительность: ≤ 500 мс (cascade) / ≤ 5 с (LLM)
- ▶ **N4.** Многоязычность: 5 рабочих языков (fr/en/de/it/es)
- ▶ **N5.** Локальное развёртывание (open-model, без внешних API)

Архитектурные (как устроено)

A1. `docker-compose`; **A2.** REST-контракт между слоями; **A3.** холодный старт без правки ядра.

В14. Слой 2: XGBoost vs MLP vs LogReg vs RF

Сравнение четырёх алгоритмов на одной выборке и одинаковом эмбединге (методология §3.3.7.5).

Классификатор	Micro-acc	Macro-F1	ECE	$t_{\text{обуч, с}}$
XGBoost (производственный)	92,02 %	0,849	0,026	1062
MLP (нейронная сеть)	92,67 %	0,857	0,031	54
Логистическая регрессия	91,85 %	0,836	0,035	8
Random Forest	89,94 %	0,809	0,027	523

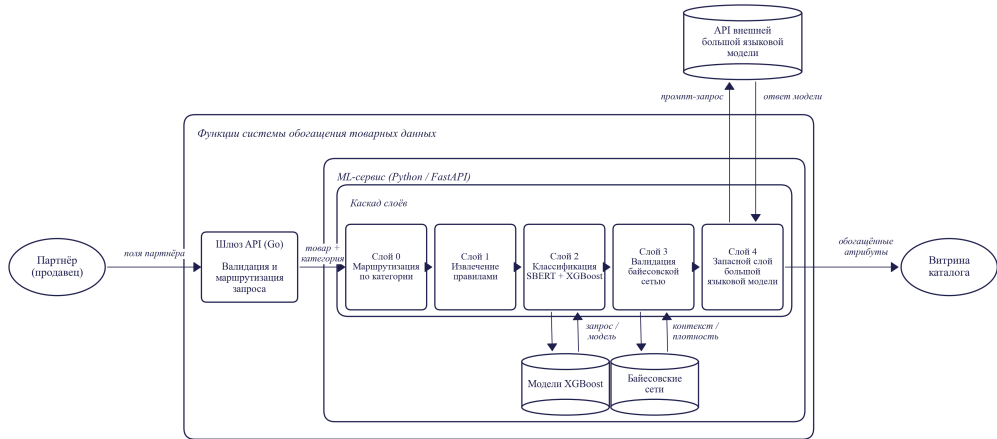
Почему XGBoost. MLP даёт чуть выше точность, но **хуже калибровку** (ECE 0,031 vs 0,026) — критично для поатрибутных порогов τ_a . RF проигрывает на 2 п. п. по точности.

Логрессия — слишком простая для разнородных категорий.

В15. Соответствие требованиям

1. **Соответствие требованиям.** Все 6 ФР, 5 НФР, 3 А реализованы; **4/4** показателя качества достигли целевых.
2. **Объём.** ~15 тыс. строк Python; 35+ модулей; 7 категориальных схем; 21 пара «категория-атрибут» (8/6/7).
3. **Программный комплекс.** `docker-compose`: React + Go + FastAPI; 5 REST-эндпоинтов; ~200 мс на запрос.
4. **Воспроизводимость.** ~70 тестов; ноутбук как исполняемый §3.3; `reproduce.sh` восстанавливает headline-цифры.

В16. Функциональная модель каскада



Каждый слой возвращает $(\hat{y}, c, \ell)$ или отказ; эскалация — только при низкой уверенности.

В17. Достоверность чисел: code-disjoint, leak-control

Достоверность

- ▶ Code-disjoint выборка: основная $n=16\,360$ + проверочная $n=566$
- ▶ **Предрегистрация** гипотезы о маршрутизаторе — `git cd9ac7a` до создания артефакта
- ▶ Wilson 95 % CI на всех headline-числах
- ▶ МакНемара + поправка Бонферрони ($\alpha/3$) для множественных сравнений
- ▶ Полное воспроизведение через `reproduce.sh`

Обоснованность. Признанные методы: sentence-transformers, XGBoost, байесовские сети, калибровка Platt + isotonic; сравнение с TXtract / MAVe / OpenTag / FrugalGPT / AutoMix.

В19. Почему один классификатор на атрибут: multitask-эксперимент

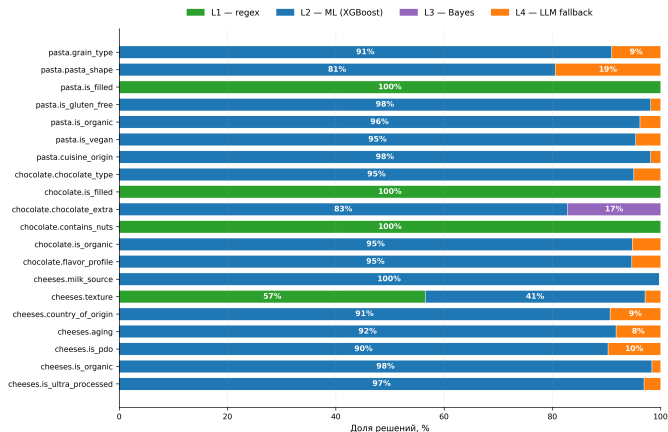
Проверено три архитектуры на 7 высоко-/средне-/слабо-коррелированных парах (3 категории):

Архитектура	Победы vs Single	Итог
MultiOutput (sklearn)	7 / 5 / 2	≈ ничья
Cartesian (декарт. пр-во)	5 / 6 / 0	—

Cartesian даёт **+0,08–0,24 macro-F1** на парах с $V \geq 0,5$ и $|A \times B| \leq 15$ (milk_source $\times$ texture: +0,235; cocoa $\times$ nuts: +0,082), но *проигрывает* на крупных пространствах (pasta_shape $\times$ is_filled, $n_{\text{class}}=22$): −0,067.

Вывод. MultiOutput технически эквивалентен независимым классификаторам (нет shared gradients). Cartesian — кандидат для пар с сильной корреляцией ($V \geq 0,5$) и узким пространством (≤ 15 классов); в производственной конфигурации оставлен per-attribute вариант как наиболее прозрачный для цикла аудит → правка →

В18. Вклад слоёв по категориям



Доля LLM по 20 атрибутам — 4,1 %; максимум на pasta.pasta_shape (19 %), pasta.grain_type (9 %). Остальное — L1 regex (22 %) + L2 ML XGBoost (73 %) + L3